

AED PARCIAL 5

Tadeo Parisato 1k2 2020

Temas Parcial 5

Análisis de algoritmos	2
Ficha 20: Análisis de algoritmos	2
Parámetros de eficiencia (factores de medición):	2
Notación Big O	2
Ficha 21: Formalización del análisis de algoritmos	4
Formalización de la notación Big O	4
Reglas de orden básicas	4
Consideraciones practicas	5
Estrategias de resolución de problemas	6
Ficha 26: Recursividad	6
Definición	6
Condiciones	6
Programación recursiva	6
Consideraciones	6
Ficha 27: Divide y vencerás – Gráficos fractales	7
Aplicaciones de la recursividad: Fractales	7
Aplicaciones de la recursión: Quicksort	7
Intercambio directo	7
Quicksort	8
La estrategia Divide y Vencerás para resolución de problemas	8
Quicksort con selección de pivot por mediana de tres	9
Función de Ackermann	9
Ficha 28: Backtracking, algoritmos ávidos y programación dinámica	9
Algoritmos deterministas	10
Fuerza bruta	10
Recursión	10
Backtracking	10
Algoritmos ávidos	10
Divide y vencerás	11
Programación Dinámica	11
Algoritmos no deterministas	11
Problema del cambio en monedas	12

Con algoritmos ávidos	12
Con programación dinámica.....	12
Aplicaciones del backtracking.....	12
Ficha 29: Teoría de la complejidad	13
Tipos de problemas.....	13
Reducción de problemas	14
Grafos.....	14
Problema del Clique Máximo.....	14
Problema del Subconjunto Independiente Máximo (SIM)	14
Máquinas Deterministas y Máquinas No Deterministas.....	14
Clases de complejidad	15
P vs NP – Problemas NP complete	16

Análisis de algoritmos

Ficha 20: Análisis de algoritmos

Parámetros de eficiencia (factores de medición): tiempo de ejecución y el espacio de memoria usado. Después en las próximas fichas entra el tema de la complejidad aparente del código fuente. El mas usado es el *tiempo de ejecución*, porque se supone que el espacio de memoria será aproximadamente el mismo en todos los algoritmos diseñados para ese problema, o que las diferencias serán apenas relevantes.

El análisis comparativo suele centrarse en dos situaciones:

- **Caso promedio:** cuando se presenta la configuración de datos más común.
- **Peor caso:** cuando se presenta la configuración de datos más desfavorable.

Lo que se busca es comparar los rendimientos relativos entre diferentes algoritmos y no medir numérica y minuciosamente el rendimiento de cada uno.

Notación Big O

Es una expresión que nos muestra una cota superior para el tiempo de ejecución. Esta forma de expresar el rendimiento de un algoritmo se puede aplicar para expresar el tiempo de ejecución, espacio, o el parámetro que se use.

Describe la complejidad del algoritmo en términos del tamaño de la entrada de datos (n) independientemente del hardware.

Busca comparar los rendimientos relativos entre los diferentes algoritmos y no tanto poder medir el tiempo.

Busca deducir alguna fórmula o expresión matemática que permita modelar el comportamiento del algoritmo en base al parámetro de eficiencia elegido.

Existen fórmulas que ya están definidas que nos permiten describir el comportamiento del algoritmo.

- **$O(1)$** : Un algoritmo con tiempo de ejecución *en el orden de uno* (o *proporcional a 1*), tiene un tiempo de ejecución *constante*, sin importar el número de datos n . Esto es claramente lo ideal al diseñar un algoritmo, pues no importa lo que crezca n , el tiempo de ejecución será siempre el mismo. Un ejemplo claro de una operación que es $O(1)$ en la práctica, es el acceso directo a un componente de un arreglo: no importa cuántos elementos tenga el arreglo (o sea, no importa el valor de n), el tiempo para acceder en forma directa a un componente es siempre el mismo.

- **$O(\log(n))$** : Un algoritmo cuyo tiempo de ejecución sea del *orden del logaritmo de n* , será ligeramente más lento a medida que n sea mayor. Es un orden muy satisfactorio en la práctica, si puede lograrse. Los algoritmos que generalmente tienen tiempos de *orden logarítmico* son los que resuelven un problema de gran tamaño procediendo a transformarlo en uno más pequeño, dividiéndolo por alguna fracción constante. Por ejemplo, el algoritmo de *búsqueda binaria* en un arreglo ordenado tiene $O(\log(n))$ en el peor caso.

- **$O(n)$** : Un algoritmo cuyo tiempo de ejecución es del *orden de n* (también se dice que su tiempo de ejecución es *lineal*), es aquél que para cada elemento de entrada realiza una pequeña cantidad de procesos iguales. Un caso típico de un algoritmo cuyo orden es *lineal*, es el de la *búsqueda secuencial en un arreglo*.

- **$O(n \cdot \log(n))$** : Se da en algoritmos que resuelven un problema dividiéndolo en pequeños subproblemas, resolviéndolos en forma independiente y combinando después las soluciones. Esta estrategia se conoce como *divide y vencerás*, y será analizada con detalle en lo que resta del curso. El algoritmo de ordenamiento *Quicksort* (en el caso promedio) tiene este rendimiento y está basado en la estrategia *divide y vencerás*.

- **$O(n^2)$** : Un algoritmo con tiempo de ejecución de *orden cuadrático*, sólo tiene utilidad práctica en problemas relativamente pequeños (o sea, con n pequeño). El tiempo de ejecución del orden de n^2 suele aparecer en algoritmos que procesan pares de elementos de datos, y la forma típica de estos algoritmos incluye un par de ciclos anidados. Se mencionó ya que todos los métodos de ordenamiento directos tienen ese tiempo de ejecución en el peor caso.

- **$O(n^3)$** : Un algoritmo de tiempo de ejecución de *orden cúbico* tampoco es muy práctico, salvo en casos de problemas muy pequeños. La forma típica de estos algoritmos incluye *tres ciclos anidados*. A modo de ejemplo, es de orden cúbico el tiempo de ejecución del popular algoritmo que permite multiplicar dos matrices entre sí.

- **$O(2^n)$** : Los algoritmos con *orden de ejecución exponencial* son muy poco útiles en la práctica. Sin embargo, aparecen con frecuencia en casos de algoritmos del tipo de *fuerza bruta*, en los que todas las soluciones posibles son investigadas una por una. Suelen aparecer tiempos de este orden en problemas de optimización de soluciones, y en esos casos se considera todo un éxito el poder replantear un algoritmo de modo de lograr tiempos cuadráticos o cúbicos...

$O(1) < O(\log(n)) < O(n) < O(n \cdot \log(n)) < O(n^2) < O(n^3) < O(2^n)$

Función	Significado (cuando mide tiempo de ejecución)	Casos típicos
$O(1)$	Orden constante. El tiempo de ejecución es constante, sin importar si crece el volumen de datos.	Acceso directo a un componente de un arreglo.
$O(\log(n))$	Orden logarítmico. Surge típicamente en algoritmos que dividen sucesivamente por dos un lote de datos, desechando una parte y procesando la otra.	Búsqueda binaria.
$O(n)$	Orden lineal. Se da cuando cada uno de los datos debe ser procesado una vez.	Búsqueda secuencial. Recorrido completo de un arreglo.
$O(n \cdot \log(n))$	Surge típicamente en algoritmos que dividen el lote de datos, procesando cada partición sin desechar ninguna, y combinando los resultados al final. No hemos analizado aún algoritmos que respondan a este orden.	Ordenamiento Rápido (Quick Sort).
$O(n^2)$	Orden cuadrático. Típico de algoritmos que combinan dos ciclos de n vueltas cada uno.	Ordenamiento por Selección Directa.
$O(n^3)$	Orden cúbico. Típico de algoritmos que combinan tres ciclos de n repeticiones cada uno. No hemos analizado aún algoritmos que respondan a ese orden.	Multiplicación de matrices.
$O(2^n)$	Orden exponencial. Algoritmos que deben explorar una por una todas las posibles combinaciones de soluciones cuando el número de soluciones crece en forma exponencial.	Problema del viajante. Solución recursiva de la Sucesión de Fibonacci.

Ficha 21: Formalización del análisis de algoritmos

Formalización de la notación Big O

La notación Big O se usa para indicar un límite superior para el comportamiento esperado de un algoritmo en cuanto al tiempo de ejecución o el espacio ocupado o algún otro parámetro.

Se intenta determinar la forma de variación de la función f de acuerdo a la variación de otra función g ya conocida. A partir de cierto punto concreto siempre f será $\leq$ a la función de la forma $c \cdot g(n)$ (Cuando n tiende a infinito tienden ser menores o iguales a alguna función de la forma $c \cdot g(n)$).

La expresión $O(g(n))$ en realidad está implicando una familia de funciones cuyo comportamiento es caracterizado por el comportamiento de g .

Lo que importa es lo que pasará para valores grandes o muy grandes de n , que es lo que se conoce como el comportamiento asintótico de la función.

En ese sentido, la expresión $O(g(n))$ es lo que se conoce como un orden de complejidad: un conjunto o familia de funciones que se comportan asintóticamente de la misma forma.

Al expresar una relación de orden en notación Big O, se está indicando una cota superior para el comportamiento de una función f en términos del comportamiento de la familia de funciones $O(g)$, pero en general el analista buscará la menor cota superior para f .

Reglas de orden básicas

En la notación Big O:

- Se puede prescindir de las constantes y quedarse solo con el término o función dominante.
- Cualquier función polinómica en n de grado k , es orden n^k .

Análisis Asintótico

No diga que un algoritmo es: $O(n^3 + n^2)$ en este caso bastará con decir $O(n^3)$

- Las constantes en los exponentes pueden obviarse.

Sea $f(n) = 2^{n+10}$ y $g(n) = 2^n \rightarrow 2^{n+10} = O(2^n)$

- la base de los logaritmos puede obviarse, ya que la función $\log_b(n)$ es $O(\log_2(n))$ para cualquier base $b > 1$.

$\log_b(n) = O(\log(n))$ [con $b > 1$]

Consideraciones prácticas

1. Dado el algoritmo que debe analizar, tenga en claro cuál será el factor de eficiencia que quiere analizar.
2. En ese algoritmo determine con exactitud el tamaño del problema (o volumen de datos a procesar).
3. En el algoritmo analizado, determine claramente cuál es la instrucción crítica que ese algoritmo lleva a cabo. La instrucción crítica es la instrucción o bloque de instrucciones que se ejecuta más veces a lo largo de toda la corrida.
4. No es lo mismo realizar un conteo exhaustivo y riguroso de operaciones críticas, que hacer un análisis asintótico.

Conteo exhaustivo: Trata de expresar una fórmula con toda rigurosidad y detalle de constantes, términos no dominantes y términos independientes que indique cuántas operaciones críticas se ejecutan exactamente para un valor dado de n .

Análisis Asintótico: Si ya tiene una expresión de conteo exhaustiva identifique el término dominante en ella, y límitese a ese término.

- Si la operación crítica (es, por ejemplo, una comparación) y está contenida dentro de dos ciclos anidados de n iteraciones cada uno, entonces su algoritmo es $O(n^2)$.
- Si su algoritmo consta de varios bloques de instrucciones independientes entre sí: su algoritmo tendrá un tiempo de ejecución en el orden del que corresponda al bloque con mayor tiempo.
- Si todo el algoritmo está compuesto sólo por un bloque de instrucciones de tiempo constante sin ciclos ni procesos ocultos dentro de una función, entonces todo el algoritmo se ejecuta en tiempo constante $t(n) = O(1)$
- Si su algoritmo se basa en tomar un bloque de n datos, dividirlo por 2, aplicar una operación de tiempo constante y luego procesar sólo una de las mitades de forma de volver a dividirla y continuar así hasta no poder hacer otra división, entonces su algoritmo ejecuta en tiempo

$t(n) = O(\log_2(n))$ que es lo mismo a:

$t(n) = O(\log(n))$ sin importar la base del logaritmo.

Estrategias de resolución de problemas

Ficha 26: Recursividad

Definición

Una definición se dice recursiva si el objeto o concepto que está siendo definido aparece a su vez en la propia definición.

Es una técnica que se basa en definir relaciones en términos de ellas mismas.

Condiciones

1. Una definición recursiva correctamente planteada, exige que la definición agregue conocimiento respecto del concepto u objeto que se define.
2. Por otra parte, una definición recursiva correctamente planteada debe evitar la recursión infinita que se produce cuando en la definición no existen elementos que permitan cerrarla lógicamente.

Programación recursiva

“Subrutinas recursivas”: una función recursiva es aquella que se invoca a sí misma una o más veces.

Para no transformarse en una función recursiva infinita se debe aplicar la condición de corte, relacionada al caso base o trivial, que es un caso que se puede resolver en forma directa.

Cuando una función es invocada (sea o no recursiva) automáticamente se asigna para ella un bloque de memoria en el segmento de memoria conocido como **Stack Segment** (o Segmento de Pila). Dentro de ese bloque, la función crea y aloja sus parámetros formales y variables locales y luego comienza a ejecutarse. Se dice que se está ejecutando una instancia de la función.

El stack segment se comporta como una pila o de tipo LIFO, del ingles stack.

Backtracking, proceso de vuelta atrás (en la ficha 28)

Lo importante de todo esto es que tanto el proceso de asignación de memoria de stack al comenzar el proceso recursivo hacia adelante (o cascada recursiva), como el proceso de vuelta atrás o backtracking, son gestionados en forma automática por el programa.

El proceso recursivo implica una cascada de invocaciones a nuevas instancias de la misma función, y luego otro proceso de regreso o vuelta atrás mediante el cual se van cerrando los cálculos que estuviesen pendientes en instancias anteriores.

Procesamiento de orden previo (pre orden): se caracteriza por el hecho de que el proceso que debe realizar la función (en este caso, el print()), se hace antes que la invocación recursiva.

Procesamiento en orden posterior (post-orden): proceso a realizar (nuevamente la función print()) se hace después de hacer la invocación recursiva.

Consideraciones

- Una función recursiva es extremadamente compacta en cuanto a código fuente
- La recursividad insume más memoria de la que usaría un proceso cíclico común. Esto se debe a que cada instancia recursiva pide memoria de stack para esa instancia. Si la

cascada recursiva fuera muy larga, podría llegar a provocarse un rebalsamiento, desborde o sobreflujo (overflow) de memoria de stack.

- La ejecución de una función requiere cierto tiempo de procesamiento desde que esta comienza y hasta que finaliza, que debe contemplarse en la estimación del tiempo total de ejecución de un proceso recursivo completo.
- Para aplicar recursividad se tienen en cuenta los tres factores generales de análisis comparativo entre algoritmos (consumo de memoria, tiempo de ejecución y complejidad de código fuente)
- En general se suele aconsejar usar la recursividad sólo cuando sea absolutamente necesario por la naturaleza implícitamente recursiva del problema que se enfrenta, recorrido de árboles, grafos, gráficos fractales, ...
 - Conviene usarla en soluciones donde no se puede usar otra técnica o en soluciones naturalmente recursivas. La complejidad del código se justifica con el consumo extra de memoria y el tiempo de ejecución será aceptable.

Ficha 27: Divide y vencerás – Gráficos fractales

Aplicaciones de la recursividad: Fractales

Una figura fractal es aquella que se compone de versiones más simples y pequeñas de la misma figura original, existen numerosos ejemplos de formaciones fractales en la naturaleza.

Muchas figuras fractales naturales pueden imitarse y reproducirse artificialmente mediante algoritmos recursivos. Pero también puede pensarse directamente en generar figuras y curvas basadas en geometría fractal que no estén inspiradas en figuras naturales.

Python provee numerosos y potentes mecanismos para el diseño de ventanas y componentes gráficos.

El sistema de coordenadas de Python permite localizar a cada punto de luz o pixel de la pantalla en forma directa.

El punto superior izquierdo dentro del área de dibujo de una ventana o un lienzo gráfico (conocido como un canvas) es el origen de coordenadas de ese contenedor gráfico, el número de filas crece hacia abajo en la pantalla. El crecimiento en el número de columnas sigue la misma idea que los gráficos cartesianos normales: crecen hacia la derecha en la pantalla.

Para crear ventanas y gráficos, usamos el módulo tkinter.

Aplicaciones de la recursión: Quicksort

Los algoritmos de ordenamiento simples o directos tienen mal rendimiento si la cantidad n de elementos de un arreglo es grande o si incluso lo que se desea es ordenar un archivo en disco.

Los algoritmos mejorados tienen un muy buen rendimiento en comparación con los simples, Se aconsejan toda vez que el arreglo sea muy grande o se quiera ordenar un conjunto en memoria externa.

Intercambio directo

- Los elementos mayores o “más pesados” tienden a viajar más rápidamente que otros “más livianos” o menores hacia su posición final del arreglo.

- El proceso de "burbujeo" que lleva a cabo ese algoritmo directo es evidentemente asimétrico, pues los elementos más grandes se comparan más veces que los más chicos en la misma pasada, haciendo que en esa misma pasada cambien de lugar también más veces que los valores pequeños.
- El resultado es que el algoritmo necesita muchas más pasadas hasta que se acomode el último de los menores que viaja desde la derecha.

Quicksort

Se caracteriza por ser el más rápido, salvo en unos pocos casos, en los que se aplican métodos para protegerse de esos casos y mantenga el buen rendimiento.

Pasos:

- 1- Se recorre el arreglo desde los dos extremos
- 2- Se toma un elemento pivot, generalmente el del medio
- 3- Luego, se recorre el arreglo desde la izquierda buscando algún valor que sea mayor que el pivot.
- 4- Al encontrarlo, se comienza una búsqueda similar pero desde la derecha, ahora buscando un valor menor al pivot.
- 5- Cuando ambos hayan sido encontrados, se intercambian entre ellos y se sigue buscando otro par de valores en forma similar, hasta que ambas secuencias de búsqueda se crucen entre ellas.
- 6- Al terminar esta pasada, se puede ver que el arreglo no queda necesariamente ordenado, pero queda claramente dividido en dos subarreglos (*el de la izquierda contiene elementos que son todos menores o iguales al pivot, y el de la derecha contiene elementos mayores o iguales al pivot*).
- 7- Ahora se puede aplicar exactamente el mismo proceso a cada subarreglo, usando recursividad hasta que sólo se obtengan particiones de tamaño uno. En ese momento, el arreglo quedará ordenado.

La estrategia Divide y Vencerás para resolución de problemas

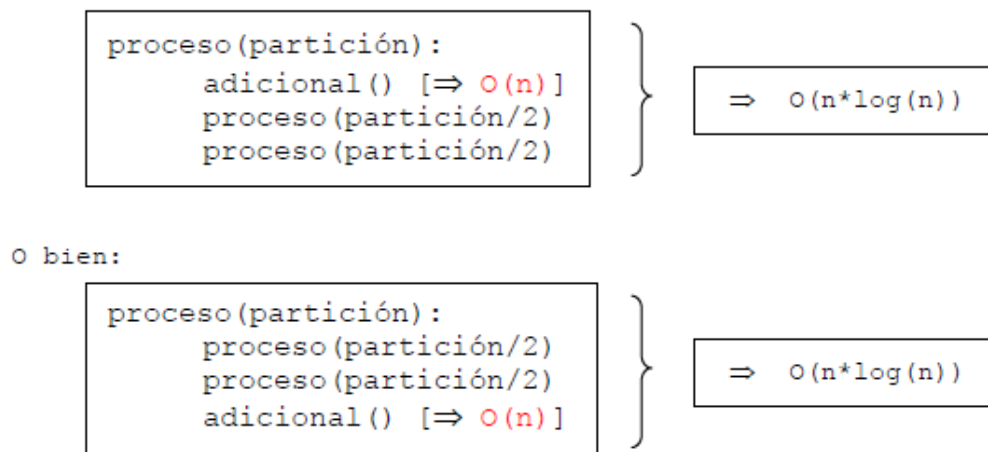
El algoritmo QuickSort es un ejemplo de aplicación exitosa de una estrategia de resolución de problema ya estudiado llamada "Divide y Vencerás", por la cual los n datos del problema estudiado se dividen en subconjuntos de aproximadamente el mismo tamaño ($n/2$ por ejemplo), luego se procesa cada uno de esos subconjuntos en forma recursiva y finalmente se "unen las partes" que se acaban de procesar para lograr el resultado final.

Con esta estrategia se puede aplicar para lograr soluciones con mejor o mucho mejor tiempo de ejecución en comparación a soluciones intuitivas de fuerza bruta (exploración de todas las posibles soluciones).

Eficiencia del Quicksort

- Pivotar: $O(n)$
- Llamadas recursivas: $O(\log(n))$
- Entonces se ejecuta en tiempo $O(n \cdot \log(n))$

Todos los algoritmos que usan la estrategia divide y vencerás, tienen tiempo de ejecución $O(n \cdot \log(n))$ siempre que el algoritmo analizado tenga la misma estructura que el algoritmo quick().



Quicksort con selección de pivot por mediana de tres

Dependiendo de cómo se tome el pivot, puede probarse que si los elementos del arreglo están distribuidos de forma tal que cada vez que se toma el pivot su valor es el menor (o el mayor) en el subarreglo considerado, entonces Quicksort ejecuta en tiempo $O(n^2)$.

Las distribuciones de datos que pueden causar que Quicksort se degrade a un rendimiento cuadrático dependen de la forma en que se selecciona el pivot. La estrategia más usada se conoce como selección del pivot por mediana de tres. Esta variante consiste en seleccionar como pivot al valor mediano entre el primero, el último y el central del subarreglo analizado.

En la implementación del Quicksort por Mediana de Tres, en cada partición se toma el valor del extremo izquierdo, el valor del extremo derecho y el valor central de la partición, se los ordena entre ellos, y se toma como pivot al que finalmente haya quedado en la casilla central.

Función de Ackermann

La Función de Ackermann se basa en aplicar un operador simple (la suma) combinado con numerosas invocaciones recursivas, con la intención de lograr un crecimiento muy rápido en la magnitud de los valores calculados.

La Función de Ackermann tiene ciertos usos prácticos: por lo pronto, debido a su naturaleza profundamente recursiva, puede utilizarse para testear la habilidad de un compilador en optimizar un proceso recursivo. En general, cuando se compila un programa recursivo, el compilador intenta eliminar la recursión y producir una versión compilada iterativa, que será más eficiente en cuanto a uso de memoria y posiblemente en velocidad de ejecución. La Función de Ackermann puede ser entonces uno de los desafíos que se le imponen a un compilador cuando esté en etapa de prueba (o benchmarking).

Ficha 28: Backtracking, algoritmos ávidos y programación dinámica

A lo largo de los años los investigadores y los especialistas en el área de los algoritmos han estudiado y propuesto diversas técnicas básicas que ayudan a plantear algoritmos para situaciones específicas. **Estas técnicas no implican una garantía de éxito**, sino simplemente un **punto de partida** para intentar encontrar una solución frente a un problema particular.

Nada garantiza que lo logre, y si lo logra, nada garantiza que la solución encontrada sea **eficiente**, pero incluso así el programador habrá tenido elementos para comenzar a trabajar y al menos poder descartar algunos caminos.

Algoritmos deterministas

La característica de un algoritmo determinista es que, ante una misma entrada, produce siempre la misma salida. Cada paso que el algoritmo aplica es una consecuencia lógica del estado en que se encontraba en el paso anterior.

Fuerza bruta

Consiste en explorar y aplicar sistemáticamente, una por una, todas y cada una de las posibles combinaciones de solución para el problema dado. Por lo general, los algoritmos obtenidos por fuerza bruta resultan intuitivamente simples de comprender e implementar. Suelen ser muy ineficientes.

Problema del Viajante o Vendedor ambulante $O(n!)$, Método de ordenamiento de Selección Directa $O(n^2)$, Búsqueda de un mayor o menor dentro de una lista $O(n)$.

Recursión

La recursión o recursividad es la propiedad que permite que un proceso se invoque a sí mismo una o más veces como parte de la solución.

Si se establece que un problema puede ser entendido en base a versiones más pequeñas y manejables de sí mismo, entonces un planteo recursivo puede ayudar a lograr un algoritmo que normalmente será muy claro para entender e implementar, al costo de utilizar cierta cantidad de memoria adicional en el segmento de stack del computador.

A pesar de los costos extra en uso de memoria y tiempo de ejecución, para algunos problemas de lógica compleja la recursión constituye una herramienta que posibilita el planteo lógico en forma clara y concisa.

Gráficos fractales, algoritmos de inserción y borrado en árboles de búsqueda equilibrados.

Backtracking

Se trata de una técnica que permite explorar en forma incremental un conjunto de potenciales soluciones (soluciones parciales) a un problema, de manera que si se detecta que una solución parcial no puede ser una solución válida, se la descarta junto a todas las candidatas que podrían haberse propuesto a partir de ella. Se aplica mediante recursión generando un árbol de invocaciones recursivas en el que cada nodo constituye una solución parcial.

Cuando es aplicable, el backtracking suele ser más eficiente que la enumeración por fuerza bruta de todas las soluciones, ya que con backtracking pueden eliminarse muchas soluciones sin tener que analizarlas.

La estrategia de backtracking se usa en problemas que admiten la idea de solución parcial

8 Reinas, palabras cruzadas, sudokus, en general en problemas de optimización combinatoria

Algoritmos ávidos

Es aquel que aplica una regla intuitivamente válida (una heurística) en cada paso local del proceso, con la esperanza de obtener finalmente una solución global óptima que resuelva el problema original. Para muchos problemas esta estrategia no produce una solución óptima,

pero suele servir de todos modos para intentar plantear soluciones óptimas locales, que luego puedan aproximar una solución óptima final en un tiempo aceptable.

La ventaja de un algoritmo ávido (cuando es correcto) es que por lo general lleva a una solución simple de entender, muy directa de implementar y razonablemente eficiente. Pero la desventaja es que como no siempre lleva a una solución correcta, se debe realizar una demonstración de la validez del algoritmo que podría no ser sencilla de hacer.

Algoritmo de Prim y de Kruskal, Algoritmo de Dijkstra, Algoritmo de Huffman

Divide y vencerás

Consiste en tratar de dividir el lote de datos en dos o más subconjuntos de tamaños aproximadamente iguales, procesar cada subconjunto por separado y finalmente unir los resultados para obtener la solución final. Normalmente, se aplica **recursión** para procesar a cada subconjunto y el proceso total podrá ser más o menos eficiente en cuanto a **tiempo de ejecución** dependiendo de tres factores: la cantidad de invocaciones recursivas que se hagan, el factor de achicamiento del lote de datos (por cuanto se divide al lote en cada pasada) y el tiempo que lleve procesar en forma separada a un subconjunto.

Quicksort y mergesort, caso promedio $O(n \cdot \log(n))$ – peor caso $O(n^2)$

Programación Dinámica

Esta técnica sugiere almacenar en una tabla las soluciones obtenidas previamente para los subproblemas que pudiera tener un problema mayor, de forma que cada subproblema se resuelva sólo una vez y luego simplemente se obtengan sus soluciones consultando la tabla si esos subproblemas volvieran a presentarse.

Se usa principalmente para resolver problemas de optimización: encontrar la mejor solución posible entre varias alternativas presentes.

Algoritmo de Bellman-Ford y de Floyd-Warshall, Algoritmos para alineación de secuencias

Algoritmos no deterministas

Cuando brindar una solución con un algoritmo determinista no es aceptable, la idea es plantear algoritmos de base aleatoria, no deterministas.

En un algoritmo no determinista, ante la misma entrada, el algoritmo podría producir salidas diferentes ya que el siguiente paso a aplicar surge de algún tipo de selección aleatoria. Y está claro que, si interviene el azar, entonces es posible que el algoritmo no llegue eventualmente a una solución correcta.

Consideraciones:

- 1- Que se justifique su uso en tiempo de ejecución.
- 2- Se espera poder conocer con precisión cuál es la probabilidad de que el algoritmo falle, y diseñarlo de tal modo que esa probabilidad sea realmente baja o muy baja.
- 3- Se esperaría que el algoritmo planteado sea relativamente simple de implementar

Problema del cambio en monedas

Con algoritmos ávidos

Posible solución: Elegir sistemáticamente la moneda de mayor valor, devolver tantas de ella como se pueda, y luego continuar así con las monedas de mayor valor que siguen hasta cubrir el valor pedido.

Requisitos:

- El vector debe estar ordenado
- Debe existir una moneda de 1
- El conjunto de monedas debe ser canónico.

No siempre funciona:

- Cuando el conjunto de monedas no es canónico (por ej con una moneda de 21 centavos) El algoritmo ávido no devuelve una solución óptima.

Con programación dinámica

Como mínimo damos monedas de 1 centavo, busco una moneda menor o igual al de x descartamos monedas mayores al monto. Buscamos la mínima cantidad de monedas en base a los resultados previamente calculados. Los nuevos resultados se almacenan.

Funciona para cualquier caso, siempre que:

- Exista la moneda de un centavo
- No es necesario que la lista de monedas esté ordenada
- No es necesario un conjunto de monedas canónico

Aplicaciones del backtracking

Problema de las 8 reinas. No deben atacarse entre ellas.

Básicamente, todas las reinas deben estar en filas y columnas diferentes, para que no se ataquen entre sí. Lo denotamos como rc (row on column, fila en columna) dentro de un arreglo unidimensional de 8 componentes.

$rc = [2, 4, 7, 0, 5, 1, 3, 6]$

Esto controla las filas y columnas, pero no las diagonales. Para estudiarlas de manera simple, identificamos dos tipos de diagonales: la diagonal principal (**diagonales normales**) y las que se orientan en sentido inverso al de la diagonal principal (**diagonales inversas**)

Propiedad de las diagonales:



- En cada una de las **diagonales normales**, la diferencia entre el número de columna y el número de fila de sus elementos es constante (15 números entre -7, 7).
- en las **diagonales inversas** es constante la suma entre el número de fila y el de columna de cada elemento (15 números diferentes entre 0 y 14).

Una matriz cuadrada tiene 15 diagonales de cada tipo, dando un total de 30 diagonales.

Con esto, agregamos 3 vectores unidimensionales más, los cuales almacenaran valores booleanos.

- **qr** (de *queens on row*) con 8 casilleros, de forma que $qr[fil] = True$ indicará que la fila *fil* está libre de reinas (o sea, no hay reinas en la fila indicada por *fil*).
- **qid** (de *queens on inverse diagonal*) con 15 casilleros, de forma que $qid[di] = True$ indicará que la diagonal inversa *di* (con $0 \leq di \leq 14$) está libre de reinas (o sea, no hay reinas en la diagonal inversa indicada por *di*).
- **qnd** (de *queens on normal diagonal*) con 15 casilleros, de forma que $qnd[dn] = True$ indicará que la diagonal normal *dn* (con $0 \leq dn \leq 14$) está libre de reinas (o sea, no hay reinas en la diagonal normal indicada por *dn*).

A la constante obtenido de una diagonal normal se le suma 7, para que en vez de estar en el intervalo $[-7;7]$, este en el intervalo $[0;14]$

Ficha 29: Teoría de la complejidad

Es la rama de las Ciencias de la Computación que estudia y clasifica a los problemas de acuerdo a su complejidad inherente. La medición de la complejidad inherente se hace con técnicas propias del análisis de algoritmos (busca determinar la cantidad de recursos que emplea un algoritmo), por lo que esta es una disciplina relacionada con la teoría de la complejidad, pero con objetivos diferentes.

La Teoría de la Complejidad intenta caracterizar a todos los posibles algoritmos para resolver un problema dado (o más precisamente, determinar qué problemas pueden o no ser resueltos empleando una cierta cantidad limitada de recursos)

Tipos de problemas

Según el objetivo:

- **Problemas de decisión:** son aquellos que sólo admiten como resultado un valor de verdad (bool).
- **Problemas de búsqueda de resultados:** son aquellos que proponen encontrar una solución específica (si existe) para el planteo dado.
- **Problemas de optimización:** son aquellos en los que se pretende encontrar la mejor solución posible (y no necesariamente cualquier solución) para el problema analizado.

Según el tiempo de ejecución:

- **Problemas inherentemente simples de resolver:** sus soluciones ejecutan en un tiempo aceptable
- **Problemas inherentemente difíciles de resolver:** sus soluciones conocidas no ejecutan en tiempo razonable, incluso para un computador potente

Consideraciones:

- Un algoritmo es razonable si puede expresarse como una función polinómica de su tamaño de entrada (tiempo de ejecución polinomial ($O(n^k)$)).
- Un tiempo polinomial es mejor o más aceptable que un tiempo exponencial o factorial, y asumir como razonable a un tiempo polinomial parece adecuado considerando las alternativas.
- **Problema Intratable:** problema para el que sólo se conocen algoritmos de tiempo exponencial, factorial o combinación entre factorial y polinomial

Reducción de problemas

Dados dos problemas t_1 y t_2 , una reducción R de t_1 a t_2 (que denotaremos como $t_2 = R(t_1)$) es un proceso por el cual las entradas del problema t_1 se convierten en entradas del problema t_2 , de forma tal que, si luego se resuelve el problema t_2 , el mismo algoritmo resuelve también t_1 .

Lo que se logra con una reducción es que un algoritmo ya planteado para resolver cierto problema, pueda aplicarse sin cambios internos para resolver otro, a condición de ajustar las entradas.

Para que una reducción entre problemas tenga sentido práctico, se esperaría obtener una ganancia en cuanto a eficiencia, es decir, el proceso para llevar a cabo la reducción debería ejecutar en un tiempo menor al que llevaría la solución del problema original. Las reducciones polinómicas podrían ser muy valiosas en casos de enfrentar problemas intratables.

Grafos

Un grafo $G = \{V, A\}$ es una estructura de datos que consta de “ V ” elementos llamados vértices y de otro conjunto “ A ” de elementos llamados arcos. Cada arco representa una relación entre dos vértices.

Cada vértice representa objetos del dominio y las relaciones entre ellos surgen del mismo dominio.

Un grafo puede implementarse de diversas formas en cualquier lenguaje de programación. Una forma muy clásica es mediante una matriz llamada matriz de adyacencias, cada fila se usa para representar un vértice de partida de un arco, y cada columna para representar un vértice de llegada.

Problema del Clique Máximo

Consiste en conocer cuál es el máximo subconjunto C de vértices de G_1 (grafo) para el cual todos los vértices “están conectados uno a uno con todos los demás” en C .

Los únicos algoritmos que se conocen para calcular un clique máximo en un grafo de n vértices, son de tiempo exponencial [$O(n^2 * 2^n)$] (mediante fuerza bruta), por lo que se trata de un problema intratable.

Problema del Subconjunto Independiente Máximo (SIM)

Se tiene un grafo G_2 y se quiere saber ahora cuál es el máximo subconjunto C de vértices de G_2 para el cual ninguno de sus vértices está conectado con los demás en C .

Ambos problemas expuestos están muy relacionados. Para rescatar la relación el G_2 es el grafo complemento inverso de G_1 (se denota: $G_2 = G_1'$) el Problema de Clique Máximo “puede reducirse” al Problema SIM (y viceversa) simplemente invirtiendo los valores de todos los casilleros de la matriz de adyacencia del grafo de entrada.

Los únicos algoritmos que se conocen son variaciones simples del mismo algoritmo de Fuerza Bruta de $O(2^n)$ que permite calcular el Clique Máximo.

Máquinas Deterministas y Máquinas No Deterministas

Existen diferentes máquinas teóricas, planteadas de manera formal para ser aplicadas en distintos escenarios y tipos de problemas.

Máquinas Deterministas: es aquella en la cual, dado su estado actual, puede determinar con certeza el siguiente estado en el cual estará en el próximo paso (Máquina de Turing).

Máquinas No Deterministas: es aquella en que no puede predecirse el estado en el que estará en el siguiente paso sólo conociendo el estado actual. Se caracterizan por elegir la respuesta correcta eligiendo en tiempo polinómico el mejor camino entre varios posibles (no es aleatoria).

Los problemas para los que ya existen buenos algoritmos en una máquina determinista, también se resolverían en forma eficiente en una no determinista: todo lo que una máquina determinista ya hace bien, también será bien resuelto por una no determinista.

Las máquinas no deterministas tendrían una ventaja: dentro de su juego de instrucciones, las no deterministas incluirían una instrucción especial que podría designarse como if-better (que podría entenderse como: si corresponde, o bien como: si es lo mejor), la cual funcionaría en forma similar a una instrucción condicional normal, pero con ciertas propiedades.

1. En forma similar a una instrucción condicional normal, la instrucción if-better dispondría de dos ramas o caminos a seguir: el mejor u óptimo y el no óptimo
2. La instrucción if_better no chequearía ninguna proposición, pero lo que sea que tuviese que hacer para determinar el próximo estado al que debiera pasar el programa, lo haría en tiempo polinómico ($O(n^k)$).
3. La instrucción if-better sería capaz de analizar el estado del programa hasta ese momento, y decidir por sí misma (sin ningún dato adicional ni proposición lógica alguna a controlar ni proceso adicional agregado por el programador) qué es lo que corresponde hacer.

Clases de complejidad

Conjunto de diversos problemas, agrupados según consumo de recursos y posibilidades de ser resueltos con diversos tipos de máquinas (teóricas o concretas)

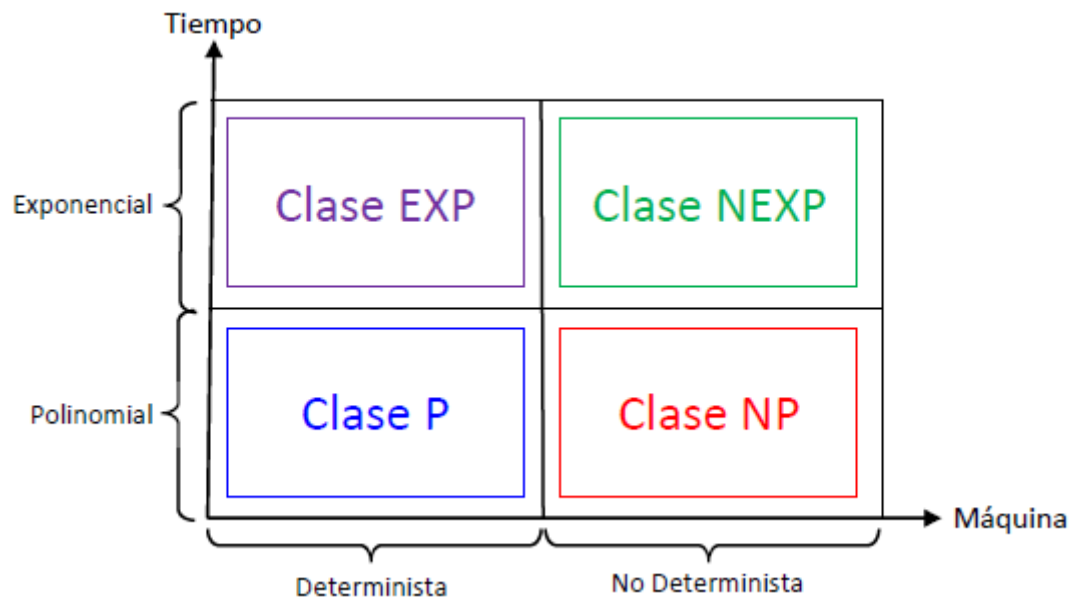
- **Clase P:** Conjunto de todos los problemas de decisión que pueden ser resueltos con algoritmos de tiempo polinomial usando una máquina determinista.

Es el conjunto de los problemas para los cuales se conocen soluciones consideradas eficientes, o al menos aceptables, desde el punto de vista del tiempo de ejecución en una máquina determinista.

- **Clase NP:** Conjunto de todos los problemas de decisión que pueden ser resueltos con algoritmos de tiempo polinomial usando una máquina no determinista.

Abarca problemas de decisión que en una máquina determinista requerirían tiempo exponencial, pero que en una máquina no determinista insumirían tiempo polinomial.

- **Clase EXP:** Conjunto de todos los problemas de decisión que pueden ser resueltos con algoritmos de tiempo exponencial usando una máquina determinista. El nombre proviene de Exponencial-Time.
- **Clase NEXP:** Conjunto de todos los problemas de decisión que pueden ser resueltos con algoritmos de tiempo exponencial usando una máquina no determinista. El nombre proviene de Nondeterministic Exponential-Time.



Además, nada impide que un mismo problema pueda ser clasificado en dos o más clases diferentes de complejidad: de hecho, es fácil ver que todos los problemas que están en P, también están en NP: si una máquina determinista puede resolver un problema en tiempo polinomial, es completamente obvio que una máquina no determinista también puede hacerlo.

$$P \subset NP \subset EXP \subset NEXP$$

P vs NP – Problemas NP complete

La clase P es subconjunto de la NP: todo problema en P pertenece también pertenece a NP. Pero la clase NP contiene muchos otros problemas que se caracterizan por ser de NP.

Los problemas que están clasificados directamente en NP y no en P se caracterizan por el hecho que no se conocen algoritmos eficientes, de tiempo polinomial, para resolverlos en una computadora normal. Pero si se encontrase una solución eficiente para un problema NP ese problema pasaría a ser P.

Si se encontrasen soluciones de tiempo polinomial para todos los problemas NP, se podría concluir que todos los problemas NP estarían también en P, es decir ($P = NP$). La realidad es que nadie sabe si la relación $P = NP$ es cierta o falsa, nadie ha podido demostrar ni lo uno ni lo otro.

El concepto de reducción polinómica intenta demostrar que $P = NP$, supongamos que:

- Se tiene un problema x que pertenece a NP y no se sabe si x pertenece a P o no.
- Además, tenemos otro problema y se sabe que pertenece a P
- Se logra encontrar una reducción polinómica R_p que transforme las entradas de x en entradas de y.
- La reducción $y = R_p(x)$ aumentó el tamaño de la clase P, pues agregó a ella un nuevo problema (x).
- Pero no podemos aplicar este procedimiento con todos los demás problemas de NP.

Supongamos que existiese en NP un problema q tan especial, que se pudiese demostrar de manera formal que todos los demás problemas de NP pueden ser reducidos a q con reducciones polinómicas.

- q podría ser la clave para probar que $P = NP$:
- Bastaría con encontrar la manera de reducir q a uno cualquiera y de los problemas de P, ya que entonces al resolver y se resolvería q, y al resolver q se resolverían todos los demás en NP.
- Estos problemas son los Problemas NP-Complete. Subconjunto de NP.

Hay dos caminos formales para demostrar que un problema q cualquiera es NP-Complete:

- a) Demostrar a través de un teorema que todo problema de NP (sea cual sea ese problema) se puede reducir tiempo en polinomial a q.
- b) Tomar un problema r del cual ya se sepa que es NP-Complete ($r \in NPC$) y reducir polinómicamente r a q (o sea, $q = R_p(r)$).

¿Cuál fue el primer problema NPComplete de todos?... Ese problema se conoce como el Problema de la Satisfactibilidad Booleana (SAT). Se tiene una fórmula o expresión compuesta por un conjunto de n variables booleanas (variables que sólo pueden valer true o false) y se quiere saber si existe alguna combinación de valores de esas variables que hagan que la fórmula entregue un resultado true.